

F21SC (Industrial Programming)

Coursework 1 - Developing a Simple Web Browser

- Shyam Sundar Velmurugan
 - Ssv2001@hw.ac.uk
 - H00418621
-

Introduction

This coursework involves developing a simple, fully functional web browser application using C# and Avalonia UI. The project focuses on implementing essential browser features such as webpage navigation, history tracking, bookmark management, and homepage configuration.

It emphasizes the integration of asynchronous HTTP communication for efficient webpage loading and the use of JSON-based file storage for maintaining user data persistently across sessions. Additionally, the application showcases an intuitive and responsive graphical user interface that allows users to interact seamlessly with multiple browsing functionalities. Overall, the project highlights a balanced combination of backend logic and user-friendly frontend design.

Features Checklist

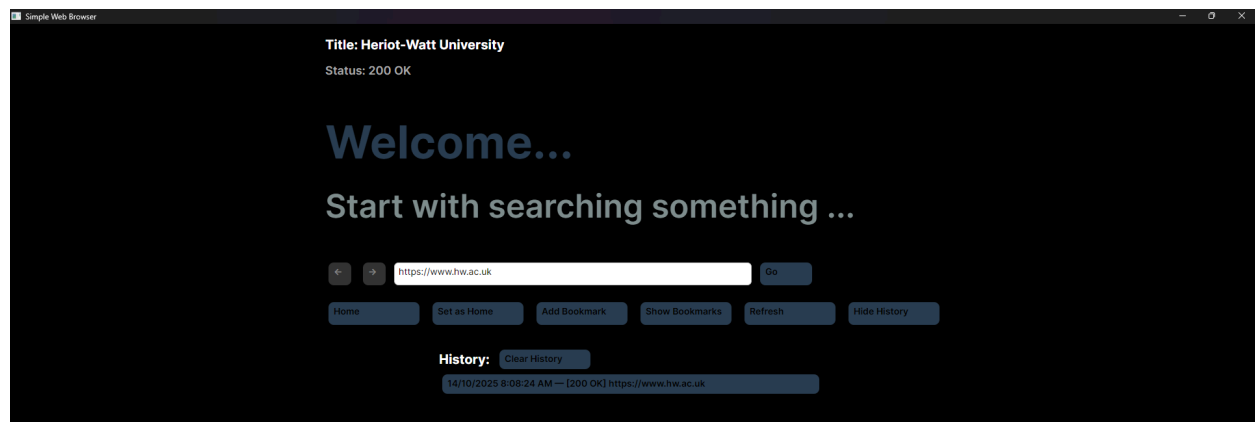
Feature	Checklist
Sending HTTP requests and displaying the contents of the response	Fully Implemented
Displaying the HTTP response status code	Fully Implemented
Displaying the webpage's title	Fully Implemented
Reload the current webpage	Fully Implemented
Setting the home page	Fully Implemented
Adding pages to bookmarks list	Fully Implemented
Naming bookmarks	Fully Implemented
Deleting bookmarks	Fully Implemented
Removing bookmarks	Fully Implemented
Navigating to bookmarks	Fully Implemented
Maintaining history	Fully Implemented
Navigating to the previous/next page in history	Fully Implemented
Navigating to a selected page in history	Fully Implemented
List five urls from the webpage in the side panel	Fully Implemented
(Advanced) Implementing database storage	Missing
(Advanced) Add multi-user functionality	Missing

Design Consideration

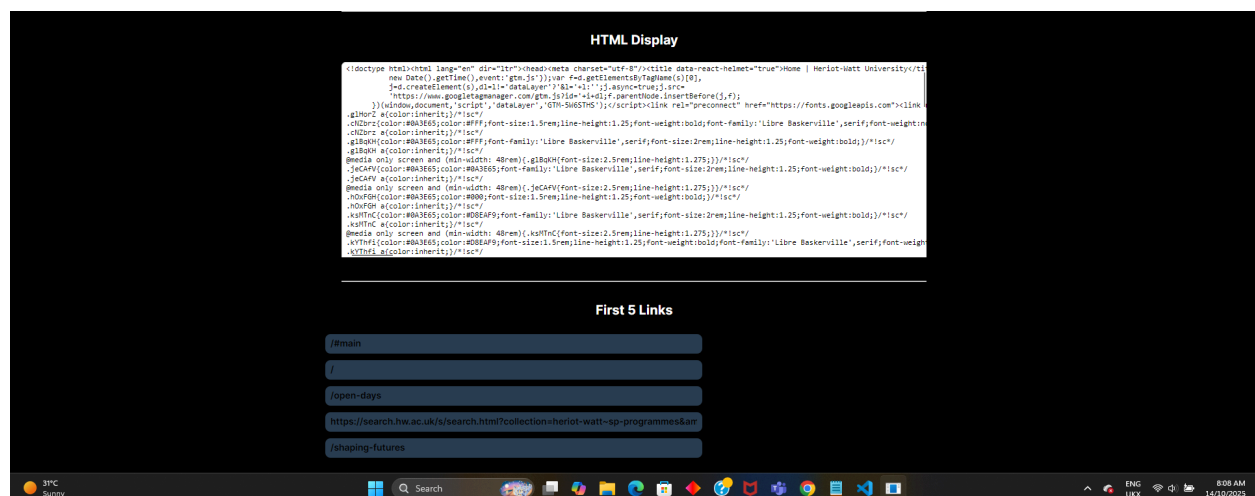
The Graphical User Interface (GUI) design:

The GUI of the web browser is primarily managed through MainWindow.axaml and MainWindow.axaml.cs. The .axaml file is responsible for defining the structural and visual components of the application, including buttons, text fields, and panels for bookmarks and history. The corresponding '.cs' file controls their behavior by handling user interactions such as clicks and navigation.

This separation of structure and logic is done for better readability and maintainability. Each UI element is named and linked to functions, ensuring simple event-driven communication between the user interface and the program's backend functionalities. It is also designed and developed using the **Avalonia UI framework**, offering a clean, modern, and responsive layout. Through XAML-based design, interactive controls such as buttons, panels, and text boxes are integrated seamlessly. Avalonia's cross-platform capabilities ensure consistent visual performance and accessibility across different operating systems.



The interface has a dark theme, making it easier for the user to read it. On the top of the web browser, whichever link is searched, the **title** of the URL and the **status** are shown. Messages like **“Welcome...”** and **“Start with searching something...”** are added to provide a friendly introduction and guide the user's actions. The **forward ‘->’, backward ‘<-’, and search button ‘go’** are kept in the same line where the URL text box is kept for easy access. The **Home** button and **Set as Home** button are used for going to the homepage URL and setting a URL as the homepage, respectively. The bookmarks are added with the **Add bookmark** button and managed with the **Show bookmarks** button. For refreshing the page, the **Refresh** button is used. History is managed and shown using the **Show History** button. Buttons are kept in uniform sizes, making it visually appealing. The buttons change both text and button color when the cursor is hovered.



For displaying the raw HTML of the searched website URL, a text box with the title **‘HTML Display’** on the top is created. The **first 5 links** present in the URL searched are added below. Between History and HTML Display, and HTML Display and the first 5 links, a grey separator line has been added.

Class Design:

Functionality Class:

The core functionality is implemented in the MainWindow class. This acts as the control center that manages navigation, history, bookmarks, and configuration. Functions like `NavigateNewAsync`, `GoBack`, and `FetchAsync` are used for handling HTTP requests, updating the displayed content, and also maintaining user interaction flow. Methods such as `LoadConfig`, `SaveHistory`, and `NormalizeUrl` ensure the organization and reduce redundancy. The division of responsibilities maintains a smooth coordination between components, allowing it to perform a specific task efficiently while maintaining simplicity for future enhancements.

Error Handling:

Try-catch blocks are used to ensure stable runtime performance and user feedback. During network communication, exceptions such as invalid URLs, unreachable servers, or timeouts are noticed and displayed as readable error messages instead of crashing the program.

For instance, reading or writing JSON files for configuration and history is also enclosed in this method of structures to prevent data corruption and maintain the continuity of user experience even when unexpected errors occur.

Displaying HTML:

The fetched HTML content is displayed within a read-only text box HTML display inside the GUI. To prevent performance issues, long HTML responses are truncated for preview. The method `RenderPage` manages this by showing only a portion of the content while also updating the title, status, and related links for clarity. Vertical and horizontal scrollbars have been added for better readability of the raw HTML extracted.

Storing Homepage, History, and Bookmarks:

The homepage, history, and bookmarks are stored and managed using JSON files. This approach is followed for easy reading, writing, and modification of data through serialization and deserialization techniques. It provides a lightweight, structured, and human-readable method for maintaining browser configurations and user preferences.

Developer Guide

(1) HTTP Request

HTTP communication is managed with the `HttpClient` library from the .NET library, providing an effective way for sending and receiving the data over the internet. The method responsible for validating the URL and providing ‘https://’ if it does not exist is `FetchAsync()`. Once it gets validated, the GET request is sent to the server using the `_httpClient.GetAsync(url)`. This is done for the application to remain as a responsive web browser until the server’s response.



The user enters a URL and clicks the ‘Go’ button. The browser sends an HTTP request using the `HttpClient` class.

HTML Display

```
@media only screen and (min-width: 48rem){.gldqnr{font-size:2.5rem;line-height:1.275;}}/*!sc*/
.jeCAfV{color:#0A3E65;color:#0A3E65;font-family:'Libre Baskerville',serif;font-size:2rem;line-height:1.25;font-weight:bold;}}/*!sc*/
.jeCAfV a{color:inherit;}}/*!sc*/
@media only screen and (min-width: 48rem){.jeCAfV{font-size:2.5rem;line-height:1.275;}}/*!sc*/
.h0xFGH{color:#0A3E65;color:#000;font-size:1.5rem;line-height:1.25;font-weight:bold;}}/*!sc*/
.h0xFGH a{color:inherit;}}/*!sc*/
.ksMTnC{color:#0A3E65;color:#D8EAF9;font-family:'Libre Baskerville',serif;font-size:2rem;line-height:1.25;font-weight:bold;}}/*!sc*/
.ksMTnC a{color:inherit;}}/*!sc*/
@media only screen and (min-width: 48rem){.ksMTnC{font-size:2.5rem;line-height:1.275;}}/*!sc*/
.kYThfi{color:#0A3E65;color:#D8EAF9;font-size:1.5rem;line-height:1.25;font-weight:bold;font-family:'Libre Baskerville',serif;font-weight:bold;}}/*!sc*/
.kYThfi a{color:inherit;}}/*!sc*/
.bWhwkZ{color:#0A3E65;color:#031B39;font-size:1.5rem;line-height:1.25;font-weight:bold;}}/*!sc*/
.bWhwkZ a{color:inherit;}}/*!sc*/
.ljiOVZ{color:#0A3E65;color:#0A3E65;font-size:1.5rem;line-height:1.25;font-weight:bold;font-family:'Libre Baskerville',serif;font-weight:bold;}}/*!sc*/
.ljiOVZ a{color:inherit;}}/*!sc*/
.kgsdVm{color:#0A3E65;font-family:'Libre Baskerville',serif;font-size:2rem;line-height:1.25;font-weight:bold;}}/*!sc*/
.kgsdVm a{color:inherit;}}/*!sc*/
@media only screen and (min-width: 48rem){.kgsdVm{font-size:2.5rem;line-height:1.275;}}/*!sc*/
.dhxhHa{color:#0A3E65;font-size:1.5rem;line-height:1.25;font-weight:bold;font-family:'Libre Baskerville',serif;font-weight:normal;font-size:1.5rem;}}/*!sc*/
.dhxhHa a{color:inherit;}}/*!sc*/
.klITEx{color:#0A3E65;color:#031B39;font-size:1.5rem;line-height:1.25;font-weight:bold;font-family:'Libre Baskerville',serif;font-weight:bold;}}/*!sc*/
.klITEx a{color:inherit;}}/*!sc*/
```

Then, the `HttpClient.GetAsync()` method will retrieve the web page HTML response, and it will display in the HTML display area below.

Title: Heriot-Watt University

Status: 200 OK

The program also finds the status code and title of the webpage and mentions it on the top.

The `ReadAsStringAsync()` method is responsible for receiving the status code for displaying meaningful feedback to the user. Finally, the `RenderPage()` method is responsible for updating the interface with the HTML content, page title, and status code. This overall ensure the smooth browsing and proper handling of both successful and error-based web responses.

(2) Homepage

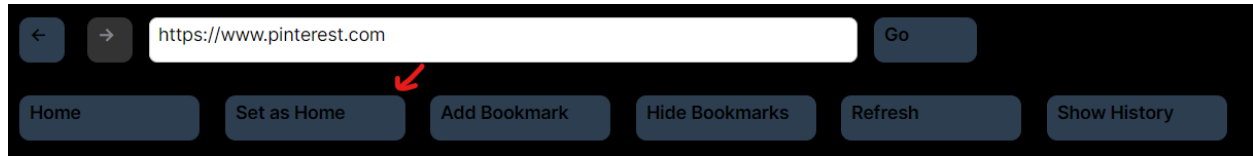
The homepage was implemented with the `config.json` that stores the homepage URL. When the application is started, with the method `LoadConfig()`, it checks if that file exists, and if yes, it returns the stored URL; otherwise, the stored default homepage (<https://www.hw.ac.uk>) is used.

Users can change their homepage with the help of the 'Set as Home' button. This button is managed with the method `SaveConfig()` for overwriting the JSON file with a new URL. This makes sure that whenever the application is started, the new updated URL only is set as the homepage URL and not the one provided previously.

Home

Set as Home

It's coded in such a way that if the Home button is clicked, it gets to the URL, which is stored as the home page: '<https://www.hw.ac.uk> here.'



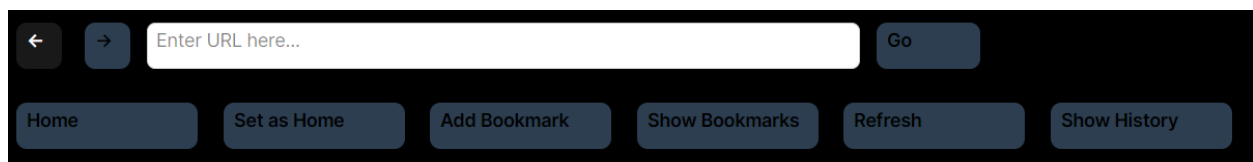
Homepage saved: https://www.pinterest.com

- If the Set as Home button is clicked, it stores the typed URL in the URL box as the new Home page URL with a line 'Homepage saved: <New URL>'.

If the single file SimpleBrowser.exe is run, it automatically generates the config.json file wherever the application file is stored.

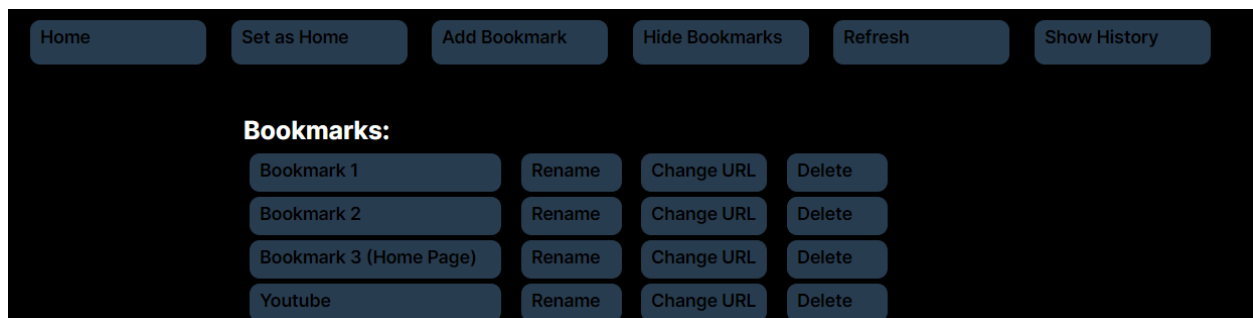
(3) Bookmarks

The bookmarks are implemented with bookmarks.json. Whenever the user adds a new bookmark using the 'Add Bookmark' button, the method AddBookmark() checks if the same URL is already present in the bookmark, ensuring no duplicate URL exists, and only then does it get stored in the list. The list is managed with JsonSerializer to make sure data is accessible whenever the application is started. The user can manage the bookmarks by modifying the name of the bookmark and the URL of that bookmark and deleting the bookmark.

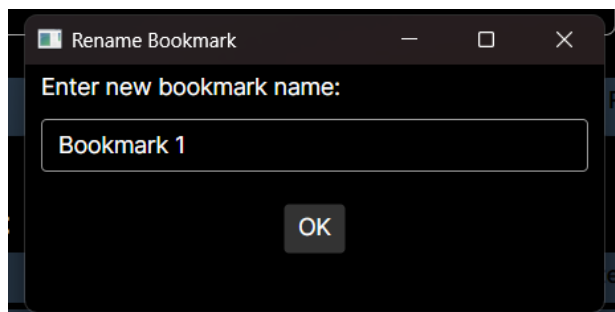


It consists of two buttons, 'Add Bookmark' and 'Show Bookmarks,' where:

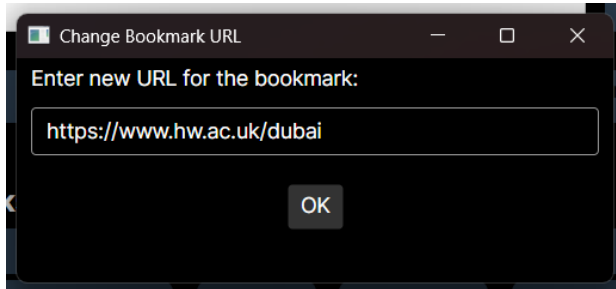
- If a URL needs to be added, the user clicks on the Add Bookmark button to store it.



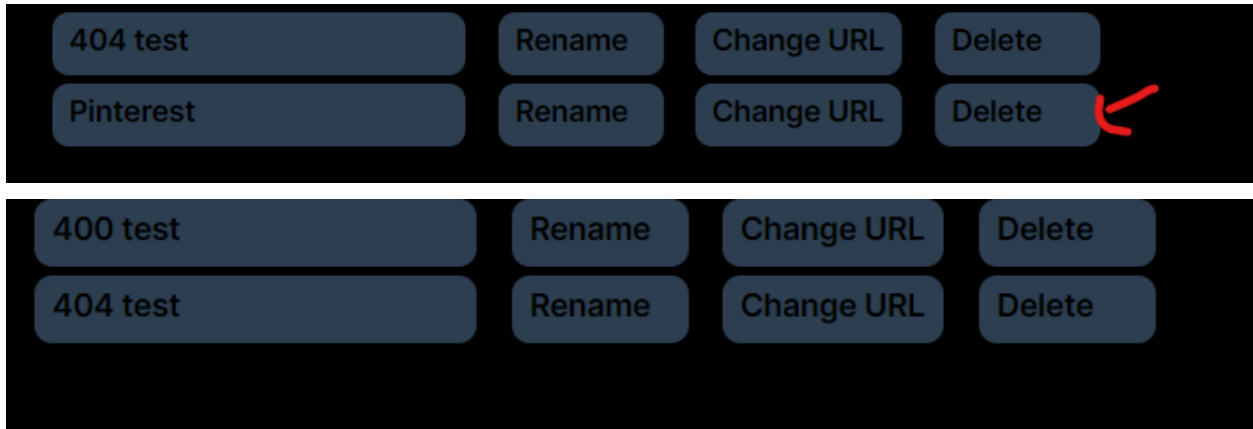
- If the user wants to visit the saved bookmarks, by clicking the Show Bookmark button, the button text turns to Hide Bookmark. It also consists of:



- Rename: Used for Renaming the Bookmark name (initially stored as Bookmark 1, Bookmark 2, ...)



- Change URL: Used for changing the URL stored into a new URL.



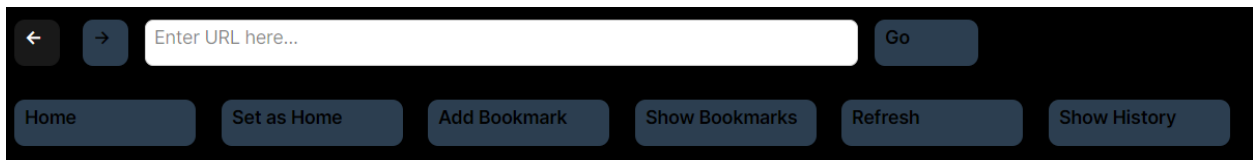
- Delete: Removes the bookmark.

If the single file SimpleBrowser.exe is run, it automatically generates the bookmark.json file wherever the application file is stored.

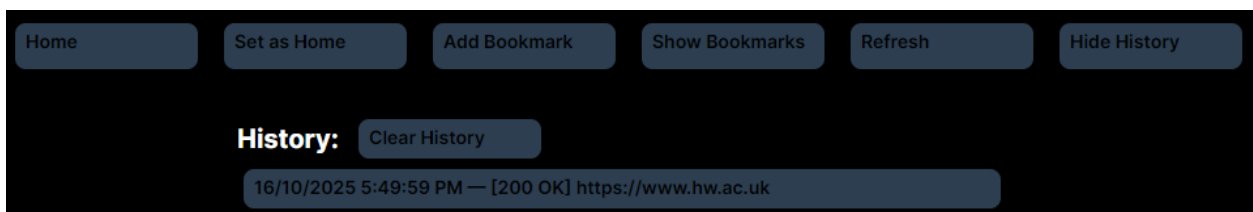
(4) History

This history feature in the web browser allows users to track and revisit the previously visited web pages. It's stored in such a way that it includes the URL, the time stamp, and the status code. Every time the user visits a new page, the AppendHistoryIfNotDuplicate() method is used, which checks the last entry. This is done to prevent the consecutive duplicates.

When the browser is loaded, the LoadHistory() method reads from the history.json file using JsonSerializer.Deserialize() for restoring the stored list of URLs. The history URLs are stored in the format of a clickable button so that the user can navigate to that particular URL directly.



If the user wants to see the history, it can be viewed by clicking on the Show History button.



- Upon clicking the Show history button, it changes to Hide history. With this we can hide the history.
 - There is a button called Clear history, which, upon clicking, clears out the stored history URL links inside the history.json file and here.
 - For easier accessibility, the stored URLs in the history are clickable. Therefore, the user can visit a specific URL link present in the history.
 - If the user navigates forward and backward, the next URL gets added into the history. But if he clicks refresh, it does not get added to the history, preventing duplicate entries.
 - History is stored in the format
`<Date DD/MM/YYYY> <Time HH/MM/SS AM/PM - <[Statuscode]> <URL>`
- If the single file SimpleBrowser.exe is run, it automatically generates the history.json file wherever the application file is stored.

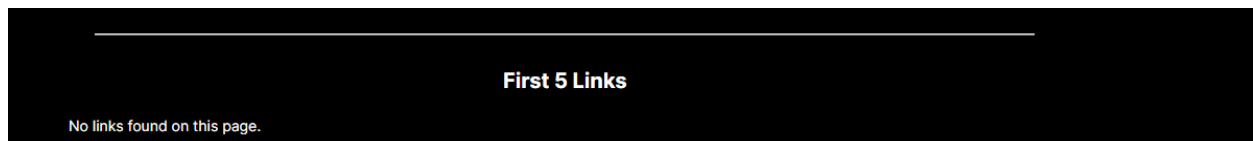
(5) First 5 URL tags

This feature automatically extracts and displays the first 5 clickable links/URLs from the currently loaded webpage. This is done using a regular expression pattern, which searches for HTML `<a>` tags with any href attributes. `Regex.Matches(html, @"<a\s+(?:[^\s]*\s+)?href\s*=\s*["'"](.*)"["'"]", RegexOptions.IgnoreCase)`

This pattern is responsible for scanning and downloading HTML for all the hyperlinks and extracts the URLs. the function `.Distinct().Take(5)` functions ensures only unique and the first five URLs are selected. Each of the extracted URL are clickable buttons inside the LinksPanel, allowing user to visit instantly.



Below the HTML Display area, the first 5 links are displayed below. These links, upon clicking, will redirect to that link and provide the very first 5 links found in that URL.



If there are no links found on the website, it displays a message saying, "No links found on this page."

Terminal codes used in CMD:

- 1) `dotnet --version`
Used it to check out what version of .NET the laptop has installed.

Terminal codes used in Visual Studio Code:

- 1) `dotnet new install Avalonia.Templates`
Used this to install the Avalonia templates.

- 2) dotnet new Avalonia.app -n SimpleBrowser
Used for generating App.axaml, App.axaml.cs, etc.
- 3) cd SimpleBrowser
Used for selecting SimpleBrowser as the current directory.
- 4) code.
Opens up the web browser application.
- 5) dotnet publish -c Release -r win-x64 --self-contained true /p:PublishSingleFile=true
/p:IncludeAllContentForSelfExtract=true /p:EnableCompressionInSingleFile=true
Used for compressing all those files into one executable file, SimpleBrowser.exe.

Extensions used:

- 1) .NET Extension Pack
- 2) .NET Install Tool
- 3) C#
- 4) C# Dev Kit
- 5) XML

How to convert the project into one executable file

- 1) To convert the project into a single executable file, open the project folder and launch the terminal. Then, run the following command:
`'dotnet publish -c Release -r win-x64 --self-contained true
/p:PublishSingleFile=true
/p:IncludeAllContentForSelfExtract=true
/p:EnableCompressionInSingleFile=true'`
- 2) After executing this command, .NET automatically generates a publish folder containing the executable file.
- 3) Inside this folder, you will find **SimpleBrowser.exe**, **SimpleBrowser.pdb**, **config.json**, and **history.json** files.
- 4) The executable file works even if it's opened in a different location without the other files
- 5) When the executable file is launched, it automatically generates **config.json** (for storing the homepage URL with "https://www.hw.ac.uk/dubai" set as the default) and **history.json** (for recording browsing history).
The **bookmark.json** file is created only after the user saves a bookmark.

Conclusion

Overall, this coursework provided an excellent opportunity to understand how to design and develop a simple yet user-friendly web browser while gaining hands-on experience with .NET, Avalonia templates, and C#. I am particularly proud of the efficient implementation of homepage storage, bookmark management with renaming functionality, and organized history tracking. However, one aspect I would have approached differently is the data storage method, specifically, replacing the use of JSON files with a database system, which would have enabled me to implement the advanced features of the coursework.
